

COMPTE RENDU TP9 PROGRAMMATION WEB

Nom: MIMBANG ATAGOUO

Prénom : Guy Théophile

1-Objectif du TP

L'objectif principal de ce TP était de rendre le formulaire de la page `contact.html` réellement interactif et dynamique en utilisant JavaScript. Le comportement final de l'interface doit se rapprocher de celui d'une application frontend moderne.

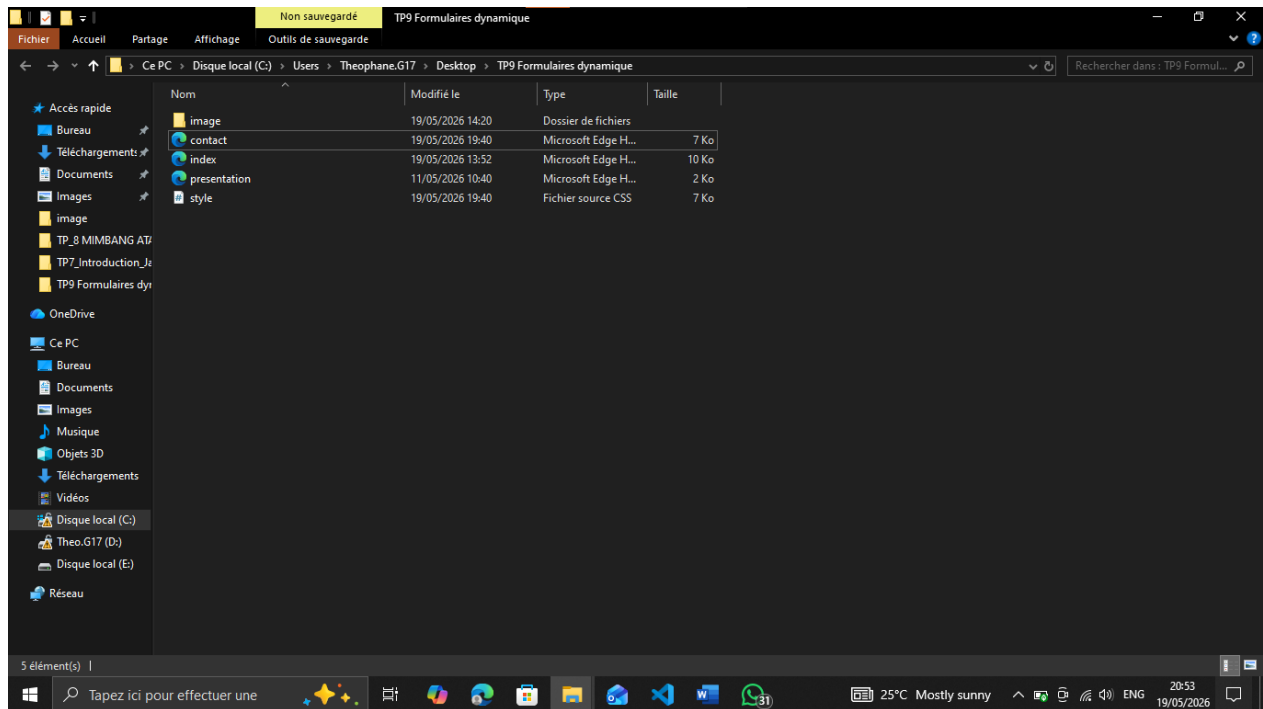
Les fonctionnalités clés validées et intégrées sont :

- **Vérification dynamique** des informations saisies par l'utilisateur au fur et à mesure.
- **Gestion visuelle des erreurs** par la modification des bordures des champs (ex : rouge en cas d'erreur, vert si valide).
- **Affichage d'un message global** clair pour résumer l'état de la validation.
- **Réinitialisation complète** du formulaire et des indicateurs visuels d'erreur lors d'un reset

2-Presentation du dossier TP9

La création du dossier TP9 ayant comme contenu les fichiers :

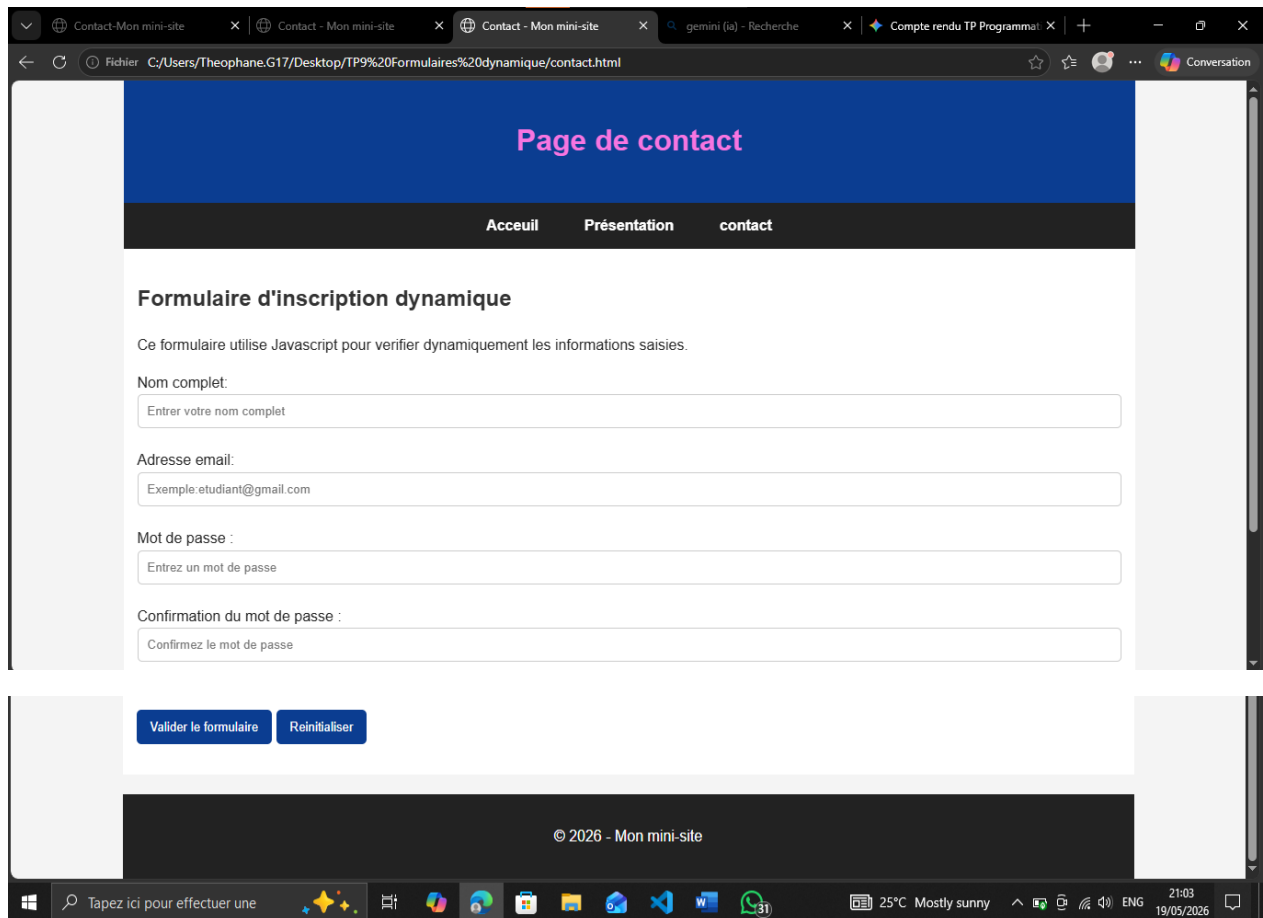
Index.html, presentation.html, contact.html et image est présentée comme suit :



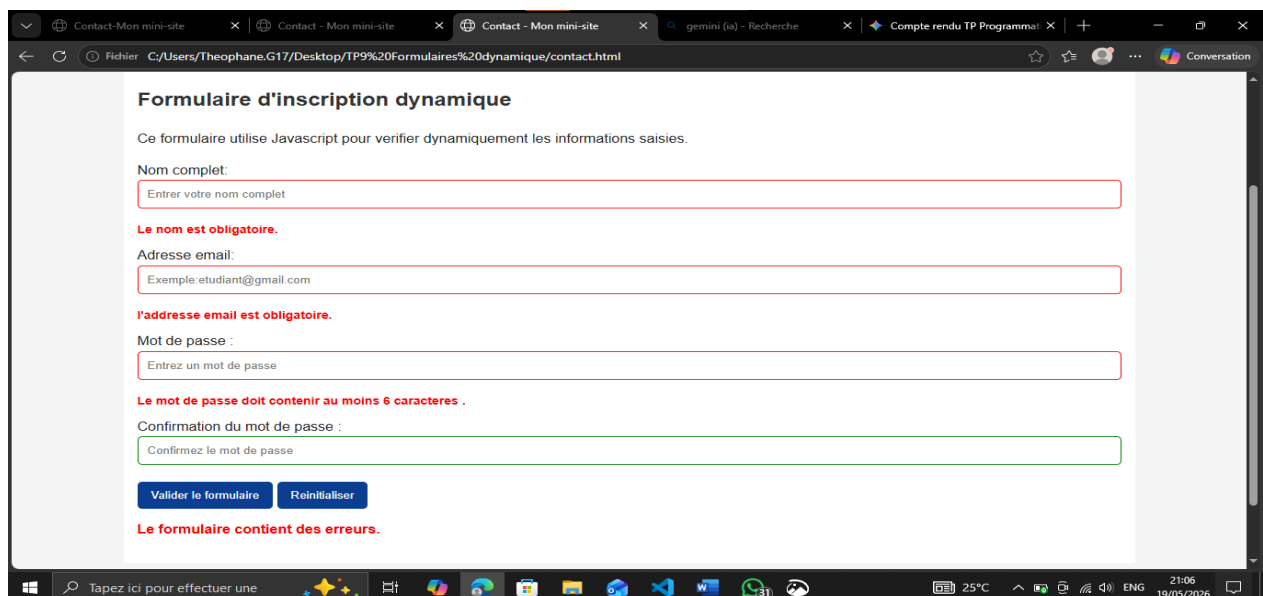
3-Presentation du travail

Pour ce TP le travail était principalement center sur la page de contact qui a connue des améliorations significatives qui ont permis à la page de contact d'être interactive. Les multiples améliorations seront respectivement présentées avec sur les images qui vont suivre :

- Page de contact sans aucune information



- Page de contact avec formulaire valide sans aucune information



- Page de contact avec les informations et le formulaire valide

Formulaire d'inscription dynamique

Ce formulaire utilise Javascript pour verifier dynamiquement les informations saisies.

Nom complet:

Adresse email:

Mot de passe :

Confirmation du mot de passe :

Le formulaire a ete valider avec succes

4-CONCLUSION

En conclusion, ce TP nous a permis de rendre un formulaire HTML interactif et dynamique grâce à JavaScript. En utilisant l'événement `oninput` et des propriétés comme `.value` et `.length`, nous avons mis en place une validation des données en temps réel. Le retour visuel instantané, via la modification de la couleur des bordures et l'affichage des messages d'erreur, a grandement amélioré l'expérience utilisateur.

5-Reponses aux questions

1. Qu'est-ce qu'une validation côté client ?

La validation côté client est une vérification des données exécutée directement dans le navigateur de l'utilisateur (grâce à HTML5 et JavaScript) avant même que le formulaire ne soit envoyé au serveur. Elle permet d'offrir un retour visuel instantané à l'internaute.

2. Pourquoi utilise-on JavaScript dans un formulaire ?

On utilise JavaScript pour :

Rendre le formulaire dynamique et interactif en temps réel.

Intercepter l'événement d'envoi (`submit`) pour empêcher l'expédition de données incorrectes.

Manipuler le CSS (modifier les bordures, afficher des messages textuels personnalisés) selon la validité des champs.

3. À quoi sert `.value` ?

En JavaScript, la propriété `.value` permet de récupérer (ou de modifier) le contenu textuel actuellement saisi par l'utilisateur dans un champ de formulaire (comme un `input text`, un `textarea`, ou un `select`).

4. Quelle différence existe-t-il entre `===` et `!==` ?

`===` est l'opérateur d'**égalité stricte**. Il renvoie `true` si les deux valeurs comparées sont identiques **et** de même type (ex: `5 === 5` est vrai, mais `5 === "5"` est faux).

`!==` est l'opérateur d'**inégalité stricte**. Il renvoie `true` si les valeurs sont différentes **ou** si leurs types sont différents. C'est le strict opposé de `===`.

5. À quoi sert `.length` ?

La propriété `.length` est utilisée pour obtenir la taille d'un élément.

Sur une chaîne de caractères (string), elle donne le **nombre de caractères** (très utile pour vérifier si un mot de passe est assez long).

Sur un tableau (array), elle donne le **nombre d'éléments** présents.

6. Pourquoi faut-il effacer les anciens messages d'erreur ?

Il faut effacer les anciens messages d'erreur pour éviter d'induire l'utilisateur en erreur. Si l'utilisateur corrige sa saisie, le message d'erreur précédent doit disparaître pour lui confirmer que le champ est désormais valide.

7. Quel est le rôle de `oninput` ?

L'événement `oninput` se déclenche de manière instantanée chaque fois que la valeur d'un champ change (à chaque fois qu'une lettre est tapée ou effacée). C'est l'outil idéal pour faire de la validation "au fil de la saisie".

8. Pourquoi modifier la couleur des bordures des champs ?

Modifier la couleur des bordures (par exemple en rouge pour une erreur et en vert pour un succès) offre une **excellente expérience utilisateur (UX)**. Cela permet à l'utilisateur de repérer en un seul coup d'œil visuel où se situent les anomalies dans le formulaire.

9. Quelle différence existe-t-il entre validation au clic et validation pendant la saisie ?

Validation au clic (souvent au `submit`) : Les vérifications se font en bloc au moment où l'utilisateur clique sur le bouton "Envoyer". Si des erreurs existent, le formulaire est bloqué.

Validation pendant la saisie (via `oninput` / `onblur`) : Les vérifications se font en temps réel, caractère par caractère. L'utilisateur sait immédiatement s'il fait une erreur, sans attendre la fin.

10. Pourquoi une validation côté serveur reste-t-elle nécessaire ?

La validation côté client est uniquement là pour le confort de l'utilisateur. Elle **n'est pas sécurisée** car le JavaScript peut être désactivé, contourné ou modifié par un utilisateur malveillant. La validation côté serveur est la seule barrière de sécurité absolue pour protéger la base de données contre les injections et les données corrompues.